COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	10/02/2026	**DATE SUBMITTED:**	10/02/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10 GRAPHS

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOA

Code

```
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head)    {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;

        newNode->next = head;    // point new node to current head
        return newNode;
    }
    int N;  // number of nodes in the graph

public:
    adjNode **head;                 //adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N)  {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;
        // initialize head pointer for all vertices
```

```
            head[i] = nullptr;
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++)  {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head
            [start_ver]);

            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }
    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
            delete[] head[i];
        delete[] head;
    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
            << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
```

```
}

// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6;      // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex,
    end_vertex, weight):"<<std::endl;

    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

Output	Graph adjacency list (start_vertex, end_vertex, weight): (0, 2, 4) (0, 1, 2) (1, 4, 3) (2, 3, 2) (3, 1, 4) (4, 3, 3)
Observations: This code utilizes node structures, arrays of pointers for adjacency lists, and functions to insert adjacent vertices to create a directed and weighted graph with pointers and linked lists. adjNode acts as a single node, and graphEdge contains the definitions for the edges. DiaGraph contains the array of pointers where each element refers to the position of the vertices. The program declares an array of 6 edges with weights and prints the list with the respective function.	

Table 10-1. Graph Implementation Utilizing Adjacency Matrix and Adjacency List.

ILOB
DFS

Code

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }
    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }
    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }
```

```
    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within
        range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
```

```
// Overloads the << operator so a graph be written directly to
a stream
// Can be used as std::cout << obj << std::endl;
template <typename U>
friend std::ostream &operator<<(std::ostream &os, const
Graph<U> &G);
```

```
private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};
```

```cpp
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in
                the stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

```cpp
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Output

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

Observations:

This code implements a graph data structure with edge lists and uses DFS to traverse through it. Edge and Graph are the main structures that store the source and destination along with their weights. The searching function performs the traversal by starting from vertex 1, uses stacks to track the vertices to visit, and places the path order to the visit_order variable. Since this structure uses a stack, it follows the LIFO sequence where the last vertex is the first one to be visited. The program runs the test function, which contains the processes of assigning 9 edges, performs the traversal, and prints the results.

Table 10-2. Implementation of Depth-First Search.

BFS

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";

        os << std::endl;
    }
    return os;
}
```

```
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
```

```
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
```

```
private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

```
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

Output

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

Observations:
This code implements an alternative method of traversal by using BFS as the method of traversal. It also uses queues for storing the vertices and visiting these vertices. Unlike the previous implementation, which does not use weights, this program has defined weights to the edges. Since this implementation uses queues, it also starts with vertex 1, processes the vertices before heading to the next level, which follows the FIFO sequence.where the neighboring vertices of vertex 1 are processed before heading to the next ones.

Table 10-3. Implementation of Breadth-First Search.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILOC

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

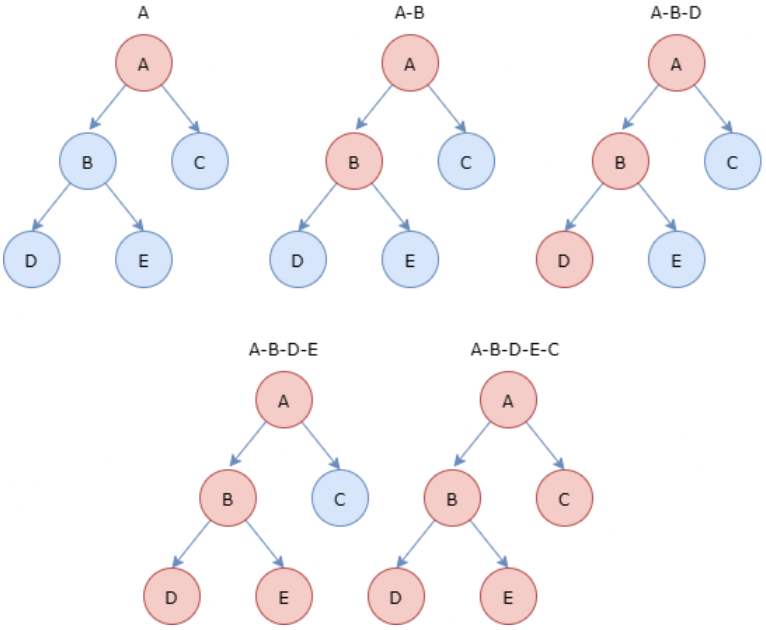
- The algorithm that would be the most helpful for this task is the Depth First Search. This algorithm starts at a defined source vertex and traverses through the lines downward until the last node is visited. Once it visits the last node, it heads back to the previous vertex that still contains nodes that have not been traversed yet, which then traverses the other available nodes. DFS is more fit for this task than BFS since DFS heads through a defined path until it visits a dead end, while BFS follows more of a level by level traversal.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

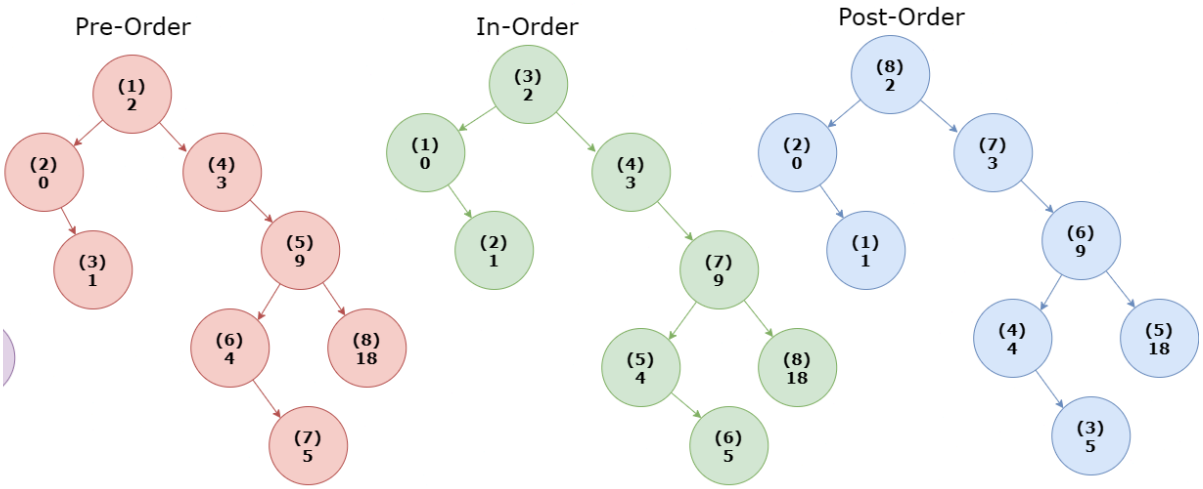
- For trees, the equivalent of DFS is the tree traversal strategies, which are pre-order, in-order, and post-order traversal. This is because these traversal methods also traverse through a single path until a dead end is visited, or in this case, a leaf node, which then heads back to the recent nodes to traverse the unvisited branches. Both follow the LIFO sequence, where they continuously traverse through the node downward until the end is visited. The contrast comes from how they work, tree strategies have a strict predefined sequence of traversing through the lists with 3 ways, while DFS generally traverses through the tree until it reaches the end point at the bottom.

GRAPHICAL COMPARISON

DFS



TREE TRAVERSAL



PSEUDOCODE	CODE IMPLEMENTATION

```
CLASS BinarySearchTree
   DECLARE key
   DECLARE left
   DECLARE right

   FUNCTION CONSTRUCTOR(value)
      key = value
      left = NULL
      right = NULL
   END FUNCTION

   FUNCTION insert(value)
      IF value < key THEN
         IF left IS NULL THEN
            left = NEW BinarySearchTree(value)
         ELSE
            left.insert(value)
         END IF

      ELSE IF value > key THEN
         IF right IS NULL THEN
            right = NEW BinarySearchTree(value)
         ELSE
            right.insert(value)
         END IF
      END IF
   END FUNCTION

   FUNCTION preOrder()
      OUTPUT key

      IF left IS NOT NULL THEN
         left.preOrder()
      END IF

      IF right IS NOT NULL THEN
         right.preOrder()
      END IF
   END FUNCTION

   FUNCTION inOrder()
      IF left IS NOT NULL THEN
         left.inOrder()
      END IF

      OUTPUT key
```

```cpp
#include <iostream>

template <typename T>
class BinarySearchTree {
private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;
```

```cpp
public:
    // Constructor
    BinarySearchTree(T value) {
        key = value;
        left = nullptr;
        right = nullptr;
    }

    // Insert a value
    void insert(T value) {
        if (value < key) {
            if (left == nullptr) {
                left = new BinarySearchTree<T>(value);
            } else {
                left->insert(value);
            }
        } else if (value > key) {
            if (right == nullptr) {
                right = new BinarySearchTree<T>(value);
            } else {
                right->insert(value);
            }
        }
    }
```

```cpp
// Preorder: Root, Left, Right
void preOrder() {
    std::cout << key << " ";

    if (left != nullptr) {
        left->preOrder();
    }

    if (right != nullptr) {
        right->preOrder();
    }
}
```

```
    IF right IS NOT NULL THEN
      right.inOrder()
    END IF
  END FUNCTION

  FUNCTION postOrder()
    IF left IS NOT NULL THEN
      left.postOrder()
    END IF

    IF right IS NOT NULL THEN
      right.postOrder()
    END IF

    OUTPUT key
  END FUNCTION
END CLASS
```

```cpp
// Inorder: Left, Root, Right
void inOrder() {
    if (left != nullptr) {
        left->inOrder();
    }

    std::cout << key << " ";

    if (right != nullptr) {
        right->inOrder();
    }
}
```

```cpp
// Postorder: Left, Right, Root
void postOrder() {
    if (left != nullptr) {
        left->postOrder();
    }

    if (right != nullptr) {
        right->postOrder();
    }

    std::cout << key << " ";
}
};
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

The data structure used to implement the Breadth First Search is a standard queue using the std::queue. It follows the FIFO (First-in, First-out) rule, where the first location to be visited is the first one to be processed by the algorithm provided. This sequence ensures the algorithm traverses through the graph in a level by level manner, which results in the traversal searching through the horizontal axis of the tree instead of a downward traversal used in DFS.

4. How many times can a node be visited in the BFS?

In the BFS, a node can only be visited once. This is because BFS traverses through the edges or nodes in a layer by layer method, which means it has to visit the surrounding nodes at the starting level before heading to the next and its following nodes. Unlike DFS, this traversal method does not support backtracking since it traverses through an x-axis and only moves through a y-axis when moving to another level.

CONCLUSION

This hands-on activity has introduced the concepts of graph nodes, which are a continuation of the previous Trees ADT laboratory task. This introduced the Depth and Breadth First searching algorithms to perform two

traversal actions in a graph. DFS traverses through the tree downwards until it reaches the end point, while BFS traverses through the tree side to side and moves on the next level. It also introduced the adjacency matrix with a mostly working C++ code implementation, which required a few tweaks to get the proper results. The procedures demonstrated these concepts by providing the working code implementations, brief information, and output and observation tables for analyzing the behavior of these algorithms. These demonstrate the differences and characteristics of BFS and DFS, while introducing new concepts in C++ programming, such as the typename U and the auto data type. The supplementary activities help expand this by comparing this to previous activities with the comparison of traversal strategies of trees and BFS and DFS of graphs. By comparing these two in terms of similarities and differences, it can help expand one's understanding and create a proper knowledge map of implementing a proper data structure with the appropriate algorithms for the given task.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation and clarification of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain the given concepts discussed in the activity and clarify the more abstract concepts in the procedures.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.